1	2	3	$\sum$

Freie Universität Berlin

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 11

Release: 06.01.2019; Due: 18.01.2019

Viktoriya Kraleva, Samuel Enderwitz

---

1 Greedy Change (__/10)

Consider the problem of making change at a cash register. We are supposed to return an amount of n cents, while using the minimum number of coins. Recall that there are Euro-coins for 1, 2, 5, 10, 20, 50, 100, and 200 cents.

(a) *Describe a greedy algorithm for the problem, and implement it. What is the running time?*

```java
import java.net.*;
import java.io.*;
import java.lang.*;
import java.util.ArrayList;
import java.util.List;

public class GreedyChange {
    //package de.smuu.add.e11;
        public static void main(String[] args) {
            int a = 389;
            List<Integer> needed = getMinCoins(a);
            System.out.println("man braucht "+needed.size() + " Muenzen naemlich: " +getMinCoins(a));

        }
        private static List<Integer> getMinCoins(int a) {
            //TODO: write with list (like alp1)
            List<Integer> neededCoins = new ArrayList<>();
            int i = 0;

            while (a > 0) {
                if (a >= 200) {
                    a -= 200;
                    neededCoins.add(200);
                } else if (a >= 100) {
                    a -= 100;
                    neededCoins.add(100);
                } else if (a >= 50) {
                    a -= 50;
                    neededCoins.add(50);
```

```
                } else if (a >= 20) {
                    a -= 20;
                    neededCoins.add(20);
                } else if (a >= 10) {
                    a -= 10;
                    neededCoins.add(10);
                } else if (a >= 5) {
                    a -= 5;
                    neededCoins.add(5);
                } else if (a >= 2) {
                    a -= 2;
                    neededCoins.add(2);
                } else if (a >= 1) {
                    a -= 1;
                    neededCoins.add(1);
                }
            }
            return neededCoins;
        }
}
```

Listing 1: GreedyChange

Wir Analsysieren die Anzahl der Vergleiche. n ist die Eingabezahl.
Da unsere Zahl beliebig groß sein kann, kann die 200cent Münze beliebig oft vorkommen. Dafür werden $\left\lfloor \frac{n}{200} \right\rfloor$ Vergleiche benötigt.
Der Betrag, dass jede Münze <200cent vorkommt ist 188. Da wir verkettet abfragen, ob die nächst kleinere Zahl reinpasst, werden mehr Abfragen gemacht, als wir Münzen benötigen, je kleiner unsere Zahl wird. Bei 188 sind das dann 2+3+4+5+6+7+8 = 20 Abfragen.
Durch die while-Schleife kommt dann nochmal für jede Münze ein Vergleich hinzu.

$$2\left\lfloor \frac{n}{200} \right\rfloor + 20 = 2\left\lfloor \frac{n}{200} \right\rfloor + 20 \in O(n)$$

Somit brauchen wir Asymptotisch so viele Vergleiche, wie groß unsere Zahl ist, also linear.

(b) *Prove that your algorithm from (a) always gives an optimal solution.*
Hint: Think about the structure of an optimal solution. How many 1-cent coins can it have at most? How many 2-cent coins? etc? What is the maximum partial amount in an optimal solution that can be represented by coins of value at most c, for $c \in \{2, 5, 10, 20, 50, 100\}$*?*

Wir betrachten erstmal die Möglichkeiten,sodass von jedem Typ mindestens eine Münze vorkommt. Wir werden auch untersuchen, wie viele Münzen am meisten vom jeden Typ vorkommen können.
Erstmal werden wir untersuchen wie viel Münzen von einem Typ am meisten vorkommen können.
Münzen von einem Cent
Davon kann man höchstens eine haben,weil wenn 2 davon vorkommen,wäre die Bediengung erfüllt, $a \geq 2$ und die wären mit einem 2 Cent Münze ersetzt.
Münzen von zwei Cent

Davon können höchstens zwei vorkommen,weil,wenn 3 davon vorkommen, wäre den gemeinsamen Wert $a = 6$. Bei 6 wird die Bediengung erfüllt $a \geq 5$ und damit wird dies nicht mit 3 Münzen von 2 Cent ersetzt,sondern mit eine Münze von 5 Cent und eine Münze von 1 Cent,weil wir unseren Algorithmus so definiert haben.

Münzen von fünf Cent

Davon darf man maximal eins haben,weil bei 2 kann man die beiden mit eine Münze von 10 Cent ersertzen. (Erklärung nicht so ausführlich,aber geht analog wie oben).

Münzen von zehn Cent

Davon darf man maximal eins haben,weil bei 2 kann man die beiden mit eine Münze von 20 Cent ersertzen. (Erklärung nicht so ausführlich,aber geht analog wie oben).

Münzen von zwanzig Cent

Davon darf man maximal zwei haben,weil bei 3 ($a = 60$) kann man die beiden mit eine Münze von 50 Cent und eine Münze von 10 Cent ersertzen,weil es so in dem Algorithmus definiert ist und es nichgt möglich wäre,diese Situation zu erwähnen. (Also der Algorithmus würde erstmal eine Münze von 50 Cent geben,weil er so definiert ist).

Münzen von fünfzig Cent

Davon darf man maximal eins haben,weil bei 2 kann man die beiden mit eine Münze von 100 Cent ersertzen. (Erklärung nicht so ausführlich,aber geht analog wie oben).

Münzen von hundert Cent

Davon darf man maximal eins haben,weil bei 2 kann man die beiden mit eine Münze von 200 Cent ersertzen. (Erklärung nicht so ausführlich,aber geht analog wie oben).

Münzen von zweihundert Cent

Wir bezeichnen mit n die Summe.Davon darf man dann $\lfloor \frac{n}{200} \rfloor$ haben. Also es hängt von der Eingabegröße ab.

Danach müssen wir die Vergleiche bei jeder Münze zählen:

Münzen von zweihundert Cent : nur ein Vergleich,weil wenn die Bediengung erfüllt ist,wird diese in der Liste hinzugefügt und man geht mit einer anderen Schleifendurchlauf weiter.

Allgemein Anzahl elementarer Schritten (Vergleichen) am meisten: $T_{200}(n) = \lfloor \frac{n}{200} \rfloor \cdot 1 = \lfloor \frac{n}{200} \rfloor$

Münzen von hundert Cent: bei jeder gefundenen Münze gibt es nur 2 Vergleiche. Erstmal für 200 Cent Münze und dann ein für 100 Cent Münze. Wie oben gezeigt,kann es dazu maximal nur eine Münze von 100 Cent geben.

Allgemein Anzahl elementarer Schritten (Vergleichen) am meisten: $T_{100}(n) = 2 \cdot 1 = 2$

Münzen von fünfzig Cent: bei jeder gefundenen Münze gibt es nur 3 Vergleiche. Erstmal für 200 Cent Münze und dann ein für 100 Cent Münze und dann ein für 50 Cent Münze,bei der die Bedingung erfüllt ist. Wie oben gezeigt,kann es dazu maximal nur eine Münze von 50 Cent geben.

Allgemein Anzahl elementarer Schritten (Vergleichen) am meisten: $T_{50}(n) = 3 \cdot 1 = 3$

Münzen von zwanzig Cent: bei jeder gefundenen Münze gibt es nur 4 Vergleiche (analog wie oben). Wie oben gezeigt,kann es dazu maximal nur zwei Münzen von 20 Cent geben.

Allgemein Anzahl elementarer Schritten (Vergleichen) am meisten: $T_{20}(n) = 4 \cdot 2 = 8$

Münzen von zehn Cent: bei jeder gefundenen Münze gibt es nur 5 Vergleiche. (ein für alle Münzen bisher und ein für die 10 Cent am Ende) Wie oben gezeigt,kann es dazu maximal nur eine Münze von 10 Cent geben.

Allgemein Anzahl elementarer Schritten (Vergleichen) am meisten: $T_{10}(n) = 5 \cdot 1 = 5$

Münzen von fünf Cent: bei jeder gefundenen Münze gibt es nur 6 Vergleiche. (analog wie oben) Wie oben gezeigt,kann es dazu maximal nur eine Münze von 5 Cent geben.
Allgemein Anzahl elementarer Schritten (Vergleichen) am meisten: $T_5(n) = 6 \cdot 1 = 6$
Münzen von zwei Cent: bei jeder gefundenen Münze gibt es nur 7 Vergleiche (analog wie oben). Wie oben gezeigt,kann es dazu maximal nur zwei Münzen von 2 Cent geben.
Allgemein Anzahl elementarer Schritten (Vergleichen) am meisten: $T_2(n) = 7 \cdot 2 = 14$
Münzen von ein Cent: bei jeder gefundenen Münze gibt es nur 8 Vergleiche. (analog wie oben) Wie oben gezeigt,kann es dazu maximal nur eine Münze von 1 Cent geben.
Dies ist auch die optimale Lösung. Allgemein Anzahl elementarer Schritten (Vergleichen) am meisten: $T_1(n) = 8 \cdot 1 = 8$
Damit wenn wir die Anzahl der Vergleichen aufsummieren,bekommen wir:
$T_n(n) = T_1(n) + T_2(n) + T_5(n) + T_{10}(n) + T_{20}(n) + T_{50}(n) + T_{100}(n) + T_{200}(n)$
$= 8 + 14 + 6 + 5 + 8 + 3 + 2 + \lfloor \frac{n}{200} \rfloor = 30 + 11 + 5 + \lfloor \frac{n}{200} \rfloor = 46 + \lfloor \frac{n}{200} \rfloor \in O(n)$ - offensichtlich warum. Damit haben wir eine lineare Laufzeit.

Ja der Algorithmus ist optimal aus dem Grund,dass unserer Algorithmus von oben nach unten ausgefüllt wird,kommen minimale Anzahl von Münzen vor,weil wir immer erstmal versuchen mit dem größeren Münze unser Zahl darzustellen. Außerdem können die Münzen $20, 2$ meistens 2 Mal vorkommen,die anderen maximal 1 Mal. Dies kann leicht überprüft werden auf folgender Weise (wir beweisen die Aussage für einen davon,für die anderen geht es analog):
Wenn die Münze 20 vorkommt. Laut unserer Algorithmus muss die Zahl $20 \leq c \leq 50$,weil andersum man bei der Überprüfung für 50 aufhören wurde.Wir nehmen eine Münze von 20 Cent und nehmen an,dass zusätzlich $c \geq 40$,weil dies den schlimmsten Fall wäre. Damit nach dem Nehmen einer Münze von 20 Cent wird: $c^l = c - 20$ und zusätzlich $c \leq 30$ und mit unserer Annahme $c \geq 20$. Mit dieser Voraussetzung nehmen wir noch eine Münze von 20 Cent und wir machen weiter mit dem Rest r. r wird (man kann dies mit Substraktion überprüfen) $0 \leq r \leq 10$ und damit werden die Münzen von 20 Cent nicht mehr unsere Voraussetzung erfüllen. Damit geht der Algorithmus weiter rekursiv,bis zu dem Zeitpunkt,wenn $r = 0$. Und wie gesagt, die optimale Lösung mit unserem Algorithmus übereinstimmen.
Analog kann man dies für alle Münzen zwischen 1 und 100 beweisen, nur bei 200 hängt dies linear mit dem Eingabewert zusammen. Damit kann man behaupten,dass unsere Lösung und die optimale Lösung bei dieser Münzenmenge übereinstimmen.

(c) *Given a family of coin denominations for which the greedy algorithm fails to give an optimal solution. There should be a 1-cent coin, so that we can make change for every possible amount.*

In Aufgabenteil (a) und (b) ist uns klar geworden, dass wir mit der dort gegebenen Münzaufteilung ein optimales Ergebnis erhalten.
Wir nehmen jetzt als Beispiel die Münzen $\{8, 5, 6, 1\}$ und wir wollen den Wert $V = 10$ daraus bekommen.
Mit unserem Algorithmus würde man 3 Münzen brauchen,nähmlich $8, 1, 1$,aber es gibt auch eine Variante, bei der man nur 2 Münzen brauchen wird. Dies wäre nähmlich

5, 5 und damit haben wir bewiesen,dass mit einer bestimmten Münzenmenge und einer bestimmten Eingabe unser Algorithmus gar nicht so effizient wäre und es wird eine bessere Lösung geben.

2 Edit Distance (__/10)

The edit distance between two strings s and t is the minimum number of edit operations needed to convert s into t. There are three edit operations: (i) insert a character; (ii) delete a character; and (iii) replace a character by another one. For example, the edit distance between "APFEL" and "PFERD" is 3: delete A, replace L by R, insert D.
Given an algorithm that determines the edit distance between two strings s and t in $O(k\ell)$ time, where s has length k and t has length ℓ. Moreover, explain how to find an optimal sequence of edit operations.
Hint: Use dynamic programming similar to the LCS-problem: consider the respective last character of s and t and distinguish three possibilities: (a) convert s to t' and add a character; (b) convert s' to t and delete a character; or (c) convert s' to t' and replace a character, if necessary. Here, s' and t' denote s and t without the last character.

1) Erstelle ein zwei Dimensionales Array mit (k+1)x(l+1)
2) Fülle die Zellen bei denen k=0 ist mit der entsprechenden Länge von l
3) Fülle die Zellen bei denen l=0 ist mit der entsprechenden Länge von k

		s_1	s_2	$\cdots$	s_k
	0	1	2	$\cdots$	k
t_1	1				
t_2	2				
$\vdots$	$\vdots$				
t_l	l				

4) Gehe alle Zeilen von links oben nach rechts unten durch
4.1) Falls Zeichen in beiden Strings gleich ist, nehme die Zahl von der Zelle schräg links oben.
4.2) Falls Zeichen unterschiedlich ist berechne, wie man den String anpassen muss, um gewünschtes Ergebnis zu erreichen und verwende dabei die "billigste" Möglichkeit (möchten kleinesten Editierabstand)
4.2.a) Zelle links -> Zeichen entfernen
4.2.b) Zelle oben -> Zeichen einfügen
4.2.c) Zelle schräg links oben -> Zeichen ändern
4.3) Schreibe in aktuelle Zelle die kleinste Zahl dieser drei und Addiere eine 1 dazu.
5) Die Zahl in (k,l) ist unser minimaler Editierabstand.

Um die optimale Sequenz der Stringedition zu erhalten, passen wir unseren Algoritmus so an, dass er in Schritt vier die Edition mit der kleinsten Zahl in einer Liste hinzufügt. Die erstellte Liste wird am Ende zurückgegeben.

```
public class EditDistanceAlgorithm {
    public static void main (String[] args){
        String str1 = "length";
        String str2 = "call";
```

```java
        int p = minimalEditDistance(str1,str2);
        System.out.println("The operations needed are: " + p);
    }


    private static int minimalEditDistance(String str1,String str2){
        int length1 = str1.length();
        int length2 = str2.length();
        int[][] myHelperArray = new int[length1+1][length2+1];
        for(int i=0;i<=length1;i++){
            for(int j =0;j<=length2;j++){
                if(i==0){
                    myHelperArray[i][j] = j;
                } else if (j==0){
                    myHelperArray[i][j] = i;
                } else if (str1.charAt(i-1)==str2.charAt(j-1)){
                    myHelperArray[i][j] = myHelperArray[i-1][j-1];
                } else {
                    int myminimum = Math.min(myHelperArray[i-1][j],Math.min(myHelperArray[i][j-1],myHelperArray[i-1][j-1]));
                        if (myminimum == myHelperArray[i - 1][j - 1]) {
                            myHelperArray[i][j] = myHelperArray[i - 1][j - 1] + 1;
                        } else if (myminimum == myHelperArray[i - 1][j]) {
                            myHelperArray[i][j] = myHelperArray[i - 1][j] + 1;
                        } else {
                            myHelperArray[i][j] = myminimum + 1;
                        }
                }
            }
        }
        printTheNeededOperations(myHelperArray,str1,str2);
        return myHelperArray[length1][length2];
    }

    private static void printTheNeededOperations(int[][] array,String str1,String str2){
        int i = str1.length();
        int j = str2.length();
        int currentnum = array[i][j];
        while(i>=1 && j >=1){
                int myminimum = Math.min(array[i-1][j],Math.min(array[i][j-1],array[i-1][j-1]));
                if (currentnum-1 == array[i - 1][j - 1]) {
                    System.out.println("Replace: " + str1.charAt(i - 1) + " with " + str2.charAt(j - 1) + " at position : " +" string 1 : "+ (i-1) +" / string2: "+(j - 1));
                    currentnum = array[i-1][j-1];
                    i -= 1;
                    j -= 1;
                } else if (currentnum - 1 == array[i - 1][j]) {
                    currentnum = array[i-1][j];
                    System.out.println("Remove: " + str1.charAt(i - 1) + " at position : " + (j - 1));
                    i -= 1;
                } else if (currentnum - 1 == array[i][j - 1] ) {
```

```
                    currentnum = array[i][j-1];
                    System.out.println("Insert: " + str2.charAt(j - 1) + " at position : " + (j - 1));
                    j -= 1;
                } else{
                    i -= 1;
                    j -= 1;
                    currentnum=array[i-1][j-1];
                }
        }
        while (j!=0){
            System.out.println("Insert: " + str2.charAt(j - 1) + " at position string 2: " + (j-1));
            j -= 1;
        }
        while (i!=0){
            System.out.println("Remove: " + str1.charAt(i - 1) + " at position string 1: " + (i - 1));
            i -= 1;
        }
    }
}
```

Listing 2: EditDistance

3 String Search (__/10)

(a) *Implement the naive algorithm for string search as well as the algorithm of Rabin-Karp. Then, determine how often the word* ***whale*** *appears in the novel* ***Moby Dick*** *(ignoring case). How do the two implementations compare?*
Hint: You can find Moby Dick under ***https://algs4.cs.princeton.edu/63suffix/mobydick.txt****.*

```
import static java.lang.Math.pow;

public class HashGenerator {
    final static int primzahl=getrandomPrimzahlinIntervall();
    private static int alphabetSignsCount ;

    public HashGenerator(){
        alphabetSignsCount=127;//Because we use ASCII
    }

    public HashGenerator(int alphabetSignsCount){
        this.alphabetSignsCount=alphabetSignsCount;
    }

    public static int hashfunktion(String string){
        int hashCode=0;
        double helper =0;
        int length = string.length();
        char c = ' ';
        for(int i = 0;i<length;i++){
            if(Character.isUpperCase(string.charAt(i))){
```

```
                c = Character.toLowerCase(string.charAt(i));
                helper += c  * pow(alphabetSignsCount,(length - 1 - i));
            } else {
                helper += string.charAt(i) * pow(alphabetSignsCount , (length - 1 - i));
            }
        }
        hashCode = (int) helper;
        hashCode %=primzahl;
        return hashCode;
    }

    public static int getrandomPrimzahlinIntervall(int lowerbound,int upperbound){
        PrimeGenerator helper = new PrimeGenerator();
        return helper.getPrime(lowerbound,upperbound);
    }
    public static int getrandomPrimzahlinIntervall(){
        PrimeGenerator helper = new PrimeGenerator();
        return helper.getPrime();
    }
}
```

Listing 3: HashGenerator

```
import java.net.*;
import java.io.*;
import java.lang.*;
import java.util.ArrayList;
import java.util.List;


public class OpenData {
    public String inputText;
    private String inputLine;
    List<Integer> myList;

    public OpenData(int alphabetSignsCount){
        this.inputText = new String();
        this.inputLine = new String();
        myList=new ArrayList<Integer>();

    }
    public OpenData(){
        this.inputText = new String();
        this.inputLine = new String();
    }
    public void setString(String str){
        this.inputText=str;
    }
    public String getString(String str){
        return this.inputText;
    }
    public List<Integer> getIndizes(){
        return myList;
    }

```

```java

    public String readFileGetText(String url) throws IOException {
       URL oracle = new URL(url);
        try {
            inputText=new String();
            BufferedReader in = new BufferedReader(new InputStreamReader(oracle.openStream()));
            while ((inputLine=in.readLine()) !=  null){
                inputText += inputLine + "\n" ;
            }
            in.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    return inputText;
    }
    public int naiveAlgorithm(String str){
        myList=new ArrayList<Integer>();
        int numberofWords = 0;
        int numberofletters=0;
        boolean upperCase=(str.charAt(0)<91 && str.charAt(0)>64);
        int length = str.length();
        String s = new String();
        char c='\0';
        int index = 0;
        for (int i =0;i<(inputText.length()-length + 1);i++){
            c = inputText.charAt(i);
            if(c==str.charAt(0) || Character.toUpperCase(c)==str.charAt(0) || Character.toUpperCase(str.charAt(0))==c){
                numberofletters=1;
                index=i;
                for (int j=1;j<length;j++){
                    if(inputText.charAt(i+j)==str.charAt(j) || Character.toUpperCase(inputText.charAt(i+j))==str.charAt(j) || Character.toUpperCase(str.charAt(j))==inputText.charAt(i+j)){
                        numberofletters++;
                    }
                }
                if (numberofletters==length){
                    myList.add((Integer) index);
                    numberofWords+=1;
                }
            }
            }
        return numberofWords;
    }

    public int rabinKarpAlgorithm(String str){
        myList=new ArrayList<Integer>();
        int numberofWords = 0;
        int length = str.length();
        int hash = HashGenerator.hashfunktion(str);
        int index = 0;
        for (int i =0;i<(inputText.length()-length);i++){
            if(hash == HashGenerator.hashfunktion(inputText.substring(i,i+length)) ){
                index=i;
```

```
                    if(equalsWithUpperCase(inputText.substring(i,i+length),
    str)){
                        myList.add((Integer) index);
                        numberofWords++;
                    }
                }
            }
            return numberofWords;
        }

        private boolean equalsWithUpperCase(String a,String b){
            if (a.length()!=b.length()){
                return false;
            }
            char c;
            int numberofletters=0;
            for (int i =0;i<a.length();i++) {
                c = a.charAt(i);
                if (a.charAt(i) == b.charAt(i) || Character.toUpperCase(a.
    charAt(i)) == b.charAt(i) || Character.toUpperCase(b.charAt(i)) == a
    .charAt(i)) {
                    numberofletters++;
                }
            }
            if (numberofletters==a.length()) {
                return true;
            } else {
                return false;
            }
        }


}
```

Listing 4: OpenData

```
import java.lang.*;
import java.util.Random;
import java.util.concurrent.ThreadLocalRandom;

public class PrimeGenerator {
    public int getPrime() {
        Random rand = new Random();
        int number = rand.nextInt(1000) + 1;

        while (isPrime(number)==false) {
            number = rand.nextInt(1000) + 1;
        }
        return number;

    }
    public int getPrime(int a,int b) {
        Random rand = new Random();
        int number = ThreadLocalRandom.current().nextInt(a, b);
        while (isPrime(number)==false) {
            number = rand.nextInt(1000) + 1;
        }
```

```java
        return number;

    }

    private static boolean isPrime(int inputNum){
        if (inputNum <= 3 || inputNum % 2 == 0)
            return inputNum == 2 || inputNum == 3; // hier ueberpruefen wir dies auch fuer alle gerade zahlen
        int divideBy = 3;
        while ((divideBy <= Math.sqrt(inputNum)) && (inputNum % divideBy != 0)) //weil wir dies schon fuer die geraden zahlen7
        //ueberprueft haben,koennen wir immer 2 aufaddieren,damit wir nur die ungeraden zahlen testen und zeit sparen.
            divideBy += 2;
        return inputNum % divideBy != 0;
    }
}
```

Listing 5: PrimeGenerator

```java
import java.io.IOException;

public class Main {


    public static void main(String[] args) throws IOException {
        OpenData file = new OpenData();
        String text;
        text = file.readFileGetText("https://algs4.cs.princeton.edu/63suffix/mobydick.txt");
        System.out.println("Robin Karp:"+file.rabinKarpAlgorithm("WHALE")+file.getIndizes()+"\n" + file.getIndizes().size());
        System.out.println("Naiv:"+file.naiveAlgorithm("WHALE") + file.getIndizes()+"\n" + file.getIndizes().size());
    }
}
```

Listing 6: Main

(b) *The algorithm of Rabin-Karp can be extended easily to several search patterns. Given a string s and search patterns $t_1, \ldots, t_k$, determine the first position in s, where one of the patterns $t_1, \ldots, t_k$ occurs. Describe how to adapt the Rabin-Karp Algorithm to this situation. What is the heuristic running time of the algorithm (assuming that collisions are rare)?*
Hint: First, assume that all search patterns have the same length. The general case can be reduced to this case, e.g., by truncating all search patterns to the same length.

Wir können es für den Fall,wo die Patterns die gleiche Länge haben so machen.
1. Split bei ‘,’ und mache eine Liste der Patterns : Damit wäre können wir die Daten in einer algemeinen Struktur speichert. **Hashwerten auswerten** : Mit Hilfe einer Iteration über unserer Liste der Patterns,werden wir den Hashwert jeder Pattern auswerten und in einem neuen Tabelle der selben Länge und die selbe Reihenfolge wie die vorherige Tabelle speichern.
Iterieren über die Buchstaben des Texts und überprüfen,ob einer der Patterns vorkommt. : Wir iterieren über der Startindex der Zeichen und überprüfen,ob

ab einer der Buchstaben ein Wort mit dem selben Hashwert vorkommt.
Pseudocode dazu:

```
def robinkarp(String s[1..i],set of Patterns){
    set hashwords =[]
    set hashvalues= []
    for sub in subs:
        hashwords.append(sub[i])
        hashvalues.append(hashfunction(sub[i])
    hashvaluestring = hashfunktion(s[0...hashwords[0].length()])
    for i from 0 to (n-hashword[0].length() +1){
        if (hashvaluestring in hashvalues){
            if s[i...i + hashwords[0].length] in hashwords{
                return i;
            }
        }
      hashvaluestring = hashfunktion(s[i+1...i+hashwords[0].length()])
      }
}
```

Mann kann der Algorithmus so bearbeiten indem man in einem zusätzlichen Struktur macht und da die Länge der Wörter und die Wörter sich selbst speichert. Danach kann mann diese Struktur darum verwenden, um die Wörter im Anhang ihrer Länge zu sortieren. Mit Hilfe dieser Trick,kann die Array die Strukturobjekten von größtem bis zu kleinsten Länge in dem Array speichern.
Dann kann mann die Schleife so umformen. Natürlich muss man der Quicksortalgorithmus ein bisschen umschreiben,damit er nur im Anhang der length Werten die Helpstrukturelemente sortiert.Wir nehmen an,dass unserer Sruktur so aussieht:

```
public class Helpstruktur{
 int length \\
 int hash \\
 String word \\
 \cdots
```

Und dann muss man den Code so umformen:

```
def robinkarp(String s[1..i],set of Patterns){
    Helpstruktur hashwords = new Belpstruktur();
    quicksort(words)
    set hashvalues= []
    int i = 0
    for sub in subs:
       hashwords[i].word.append(sub[i])
       hashwords[i].hash.append(hashfunction(sub[i])
       hashwords[i].length.append(sub[i].length)
       i++
    quicksort(hashwords)
    hashvaluestring = hashfunktion(s[0...hashwords[0].length()])
    for i from 0 to (n-hashword[0].length() +1):
      for j from 0 to hashwords.length():
      hashvaluestring = hashfunktion(s[i...hashwords[j].length()])
```

```
        if (hashvaluestring in hashwords.hash){
            if s[i...i + hashwords[j].length] in hashwords.words{
                return i;
            }
        }
}
```

(c) *(voluntary, 5 extra points) Implement the algorithm from (b). Then, determine who occurs most often in Moby Dick: Ishmael, Queequeg or Ahab?*